

Travaux Pratiques

Module M214 - Application Cloud Native

Séance 2 : API CRUD avec Mongoose

PARTIE 1 : Atelier Guidé

Créez une API de gestion de produits.

Étape 1 : Initialiser le projet

```
1 mkdir api-produits
2 cd api-produits
3 npm init -y
4 npm install express mongoose
```

Étape 2 : Créer le modèle

Créez `models/Produit.js` :

```
1 const mongoose = require('mongoose');
2
3 const produitSchema = new mongoose.Schema({
4   nom: { type: String, required: true, trim: true },
5   prix: { type: Number, required: true, min: 0 },
6   stock: { type: Number, default: 0 },
7   categorie: { type: String, required: true }
8 }, { timestamps: true });
9
10 module.exports = mongoose.model('Produit', produitSchema);
```

Étape 3 : Créer le serveur

Créez `index.js` :

```
1 const express = require('express');
2 const mongoose = require('mongoose');
```

```
3 const Produit = require('./models/Produit');
4
5 const app = express();
6 app.use(express.json());
7
8 mongoose.connect('mongodb://localhost:27017/shop')
9   .then(() => console.log('MongoDB connecte'));
10
11 app.listen(3000, () => console.log('Serveur sur port 3000'));
```

≡ Étape 4 : Routes GET

```
1 // GET tous les produits
2 app.get('/api/produits', async (req, res) => {
3   try {
4     const produits = await Produit.find();
5     res.json(produits);
6   } catch (error) {
7     res.status(500).json({ error: error.message });
8   }
9 });
10
11 // GET un produit par ID
12 app.get('/api/produits/:id', async (req, res) => {
13   try {
14     const produit = await Produit.findById(req.params.id)
15     ;
16     if (!produit) return res.status(404).json({ error: '
17 Non trouve' });
18     res.json(produit);
19   } catch (error) {
20     res.status(500).json({ error: error.message });
21   }
22 });
```

+ Étape 5 : Route POST

```
1 app.post('/api/produits', async (req, res) => {
2   try {
3     const produit = await Produit.create(req.body);
4     res.status(201).json(produit);
5   } catch (error) {
6     res.status(400).json({ error: error.message });
7   }
8 });
```

```
8 });
```

Test Postman : POST <http://localhost:3000/api/produits>

```
1 { "nom": "Laptop", "prix": 8500, "stock": 10, "categorie": "
  Informatique" }
```

Étape 6 : Route PUT

```
1 app.put('/api/produits/:id', async (req, res) => {
2   try {
3     const produit = await Produit.findByIdAndUpdate(
4       req.params.id, req.body, { new: true,
      runValidators: true }
5     );
6     if (!produit) return res.status(404).json({ error: '
Non trouve' });
7     res.json(produit);
8   } catch (error) {
9     res.status(400).json({ error: error.message });
10  }
11 });
```

Étape 7 : Route DELETE

```
1 app.delete('/api/produits/:id', async (req, res) => {
2   try {
3     const produit = await Produit.findByIdAndDelete(req.
      params.id);
4     if (!produit) return res.status(404).json({ error: '
Non trouve' });
5     res.json({ message: 'Supprime' });
6   } catch (error) {
7     res.status(500).json({ error: error.message });
8   }
9 });
```

PARTIE 2 : Exercice Autonome

API Étudiants

Créez une API pour gérer des étudiants avec les champs : **nom**, **prenom**, **email** (unique), **age** (min : 16), **filiere**, **moyenne** (0-20).

- Q1.** Écrivez le schéma Mongoose complet pour l'étudiant.
- Q2.** Écrivez la route **GET** `/api/etudiants`.
- Q3.** Écrivez la route **GET** `/api/etudiants/:id` avec gestion 404.
- Q4.** Écrivez la route **POST** `/api/etudiants`.
- Q5.** Écrivez la route **PUT** `/api/etudiants/:id`.
- Q6.** Écrivez la route **DELETE** `/api/etudiants/:id`.
- Q7.** Écrivez une route **GET** `/api/etudiants/filiere/:nom` qui retourne les étudiants d'une filière.
- Q8.** Complétez : Créer _____ | Lire tous _____ | Lire un _____ | Modifier _____ | Supprimer _____

PARTIE 3 : Challenge

Système de Gestion de Commandes E-Commerce

Une startup e-commerce vous demande de créer l'API backend pour gérer les commandes. Le système doit gérer trois entités : **Client**, **Produit**, et **Commande**.

Spécifications :

- **Client** : nom, email (unique), telephone, adresse (ville, rue, codePostal), dateInscription
- **Produit** : nom, prix, stock, categorie (enum : Electronique, Vetements, Alimentaire, Autres)
- **Commande** : client (référence), produits (tableau de {produit, quantite}), total, statut (enum : EnAttente, Validee, Expediee, Livree, Annulee), dateCommande

- Q1.** Écrivez les trois schémas Mongoose (Client, Produit, Commande).
- Q2.** Écrivez la route **POST** `/api/commandes` qui crée une commande, vérifie le stock de chaque produit, décrémente le stock, et calcule le total automatiquement.
- Q3.** Écrivez la route **GET** `/api/commandes/:id` qui retourne une commande avec les informations complètes du client et des produits (populate).
- Q4.** Écrivez la route **PUT** `/api/commandes/:id/statut` qui met à jour uniquement le statut d'une commande. Si le statut passe à "Annulee", le stock des produits doit être restauré.
- Q5.** Écrivez la route **GET** `/api/clients/:id/commandes` qui retourne toutes les commandes d'un client triées par date décroissante.
- Q6.** Écrivez la route **GET** `/api/stats/produits-populaires` qui retourne les 5 produits les plus commandés (utiliser aggregate ou une logique personnalisée).
- Q7.** Comment gérer le cas où deux utilisateurs commandent le même produit simultanément et le stock est insuffisant pour les deux ?